

Referencia completa de API

12.2. Anexo de documentación API

Sistema de Reserva Inteligente de Restaurantes — Etapa 3

Bases de Datos II · I Semestre 2026 · Prof. Kenneth Obando

Tabla de contenido

- Introducción 4
- 1. Autenticación 5
 - Resumen de endpoints 5
 - POST /api/auth/register 5
 - POST /api/auth/login 5
- 2. Usuarios 7
 - Resumen de endpoints 7
 - GET /api/users/me 7
 - PUT /api/users/:id 7
 - DELETE /api/users/:id 8
- 3. Restaurantes 9
 - Resumen de endpoints 9
 - POST /api/restaurants 9
 - GET /api/restaurants 9
- 4. Menús 10
 - Resumen de endpoints 10
 - POST /api/menus 10
 - GET /api/menus/:id 10
 - PUT /api/menus/:id 11
 - DELETE /api/menus/:id 11
- 5. Reservaciones 12
 - Resumen de endpoints 12
 - POST /api/reservations 12
 - DELETE /api/reservations/:id 12
- 6. Pedidos 13
 - Resumen de endpoints 13
 - POST /api/orders 13
 - GET /api/orders/:id 14
- 7. Búsqueda 15
 - Resumen de endpoints 15
 - GET /search/products?q=texto 15
 - GET /search/products/category/:categoria 15
 - POST /search/reindex 15
- 8. Grafos (Neo4J) 17
 - Resumen de endpoints 17
 - GET /grafos/co-compras 17
 - GET /grafos/usuarios-influyentes 17

2

GET /grafos/camino-minimo.....	17
9. Rutas de entrega.....	19
Resumen de endpoints	19
POST /rutas/asignar.....	19
10. Códigos de error comunes	21
11. Flujo típico de uso	22

Introducción

Este documento es el segundo anexo de la documentación del proyecto – etapa 3. Contiene la referencia completa de todos los endpoints expuestos por los microservicios del sistema, incluyendo campos de entrada, validaciones, formatos de respuesta, códigos de error y ejemplos de uso.

El sistema expone tres grupos de endpoints:

- API principal (CRUD/Entidades) — prefijo /api/ via Nginx en el puerto 80
- Microservicio de búsqueda — prefijo /search/ via Nginx en el puerto 80
- Microservicio de grafos y rutas — prefijo /grafos/ y /rutas/ via Nginx, o directo en el puerto 5050

Autenticación:

Todos los endpoints protegidos requieren el header Authorization: Bearer <access_token>. El token se obtiene desde Keycloak (ver sección 1 de este anexo). Los tokens expiran a los 300 segundos.

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5...
```

1. Autenticación

Antes de usar cualquier endpoint protegido hay que obtener un token JWT de Keycloak. Estos son los únicos dos endpoints que no requieren autenticación.

Resumen de endpoints

Método	Endpoint	Descripción	Auth
POST	/api/auth/register	Registro de nuevo usuario	No
POST	/api/auth/login	Login y obtención de JWT	No

POST /api/auth/register

Crea un nuevo usuario en Keycloak y lo registra en la base de datos del sistema. Solo un administrador puede crear otro administrador.

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
username	string	Sí	Nombre de usuario único en el sistema
email	string	Sí	Correo electrónico válido
password	string	Sí	Contraseña (mínimo 8 caracteres recomendado)
firstName	string	No	Nombre del usuario
lastName	string	No	Apellido del usuario
role	string	No	cliente (default) o admin. Solo un admin puede crear otro admin.

Respuestas

Código	Descripción	Body
201	Registro exitoso	{ "message": "Usuario registrado correctamente" }
400	Faltan campos obligatorios	{ "error": "username, email y password son obligatorios" }
403	Intento de crear admin sin permisos	{ "error": "Solo admins pueden crear otros admins" }
500	Error interno del servidor	{ "error": "descripción del error" }

POST /api/auth/login

Autentica al usuario contra Keycloak y devuelve un token JWT de acceso. El token tiene una duración de 300 segundos (5 minutos).

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
username	string	Sí	Nombre de usuario

Campo	Tipo	Obligatorio	Descripción
password	string	Sí	Contraseña del usuario

Respuestas

Código	Descripción	Body
200	Login exitoso	{ "access_token": "eyJ...", "expires_in": 300, "token_type": "Bearer" }
400	Faltan credenciales	{ "error": "username y password son obligatorios" }
401	Credenciales inválidas	{ "error": "Credenciales inválidas" }

2. Usuarios

Endpoints para consultar y gestionar usuarios del sistema. La eliminación y actualización de usuarios está restringida al rol admin.

Resumen de endpoints

Método	Endpoint	Descripción	Rol requerido
GET	/api/users/me	Obtener perfil del usuario autenticado	Cualquiera
PUT	/api/users/:id	Actualizar datos de un usuario	admin
DELETE	/api/users/:id	Eliminar un usuario del sistema	admin

GET /api/users/me

Devuelve el perfil del usuario identificado por el token JWT. No requiere parámetros adicionales.

Respuestas

Código	Descripción	Body
200	Perfil obtenido exitosamente	{ "id": 1, "nombre": "Juan Pérez", "correo": "juan@example.com", "rol": "cliente" }
401	Token inválido o expirado	{ "error": "No autorizado" }
404	Usuario no encontrado	{ "error": "Usuario no encontrado" }

PUT /api/users/:id

Actualiza el nombre o correo de un usuario. Se requiere al menos uno de los dos campos.

Parámetros de URL

Parámetro	Tipo	Descripción
id	número	ID interno del usuario a actualizar

Request body (application/json) — al menos un campo obligatorio

Campo	Tipo	Obligatorio	Descripción
nombre	string	Condicional	Nuevo nombre del usuario
correo	string	Condicional	Nuevo correo electrónico

Respuestas

Código	Descripción	Body
200	Actualización exitosa	{ "message": "Usuario actualizado correctamente", "data": { "id": 1, "nombre": "...", "correo": "..." } }

Código	Descripción	Body
400	No se envió ningún campo	{ "error": "Se requiere al menos nombre o correo" }
403	Sin permisos de admin	{ "error": "Acceso denegado" }
404	Usuario no existe	{ "error": "Usuario no encontrado" }

DELETE /api/users/:id

Elimina un usuario del sistema. No requiere body.

Respuestas

Código	Descripción	Body
200	Eliminación exitosa	{ "message": "Usuario eliminado correctamente" }
403	Sin permisos de admin	{ "error": "Acceso denegado" }
404	Usuario no existe	{ "error": "Usuario no encontrado" }

3. Restaurantes

Gestión de los restaurantes del sistema. La creación está restringida a administradores; la consulta está disponible para cualquier usuario autenticado.

Resumen de endpoints

Método	Endpoint	Descripción	Rol requerido
POST	/api/restaurants	Crear un nuevo restaurante	admin
GET	/api/restaurants	Listar todos los restaurantes	Cualquiera

POST /api/restaurants

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
nombre	string	Sí	Nombre del restaurante
direccion	string	No	Dirección física del restaurante
telefono	string	No	Número de teléfono de contacto

Respuestas

Código	Descripción	Body
201	Creación exitosa	{ "message": "Restaurante registrado correctamente", "id": 1 }
400	Falta el nombre	{ "error": "El nombre del restaurante es obligatorio" }
403	Sin permisos de admin	{ "error": "Acceso denegado" }

GET /api/restaurants

No requiere parámetros. Devuelve un array con todos los restaurantes. Si no hay ninguno, devuelve un array vacío.

Respuesta 200

```
[ { "id": 1, "nombre": "La Trattoria", "direccion": "Av. Central", "telefono": "2222-3333" }, ... ]
```

4. Menús

Gestión de los menús de cada restaurante. Cada menú pertenece a un restaurante y contiene una lista de platos. La consulta está disponible para usuarios autenticados; la creación, modificación y eliminación requieren rol admin.

Resumen de endpoints

Método	Endpoint	Descripción	Rol requerido
POST	/api/menus	Crear un nuevo menú	admin
GET	/api/menus/:id	Obtener un menú por ID	Cualquiera
PUT	/api/menus/:id	Actualizar nombre del menú	admin
DELETE	/api/menus/:id	Eliminar un menú	admin

POST /api/menus

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
nombre	string	Sí	Nombre del menú
id_restaurante	número	Sí	ID del restaurante al que pertenece el menú

Respuestas

Código	Descripción	Body
201	Menú creado exitosamente	{ "message": "Menú creado correctamente", "id": 1 }
400	Faltan campos obligatorios	{ "error": "nombre e id_restaurante son obligatorios" }
403	Sin permisos de admin	{ "error": "Acceso denegado" }

GET /api/menus/:id

Devuelve el menú con su lista de platos completa.

Respuesta 200

```
{ "id": 1, "nombre": "Menú del día", "id_restaurante": 1,
  "platos": [{ "id": 1, "nombre": "Casado", "precio": 4500,
    "categoria": "Típico", "descripcion": "..."}] }
```

Respuestas de error

Código	Descripción	Body
404	Menú no encontrado	{ "error": "Menú no encontrado" }
401	No autenticado	{ "error": "No autorizado" }

PUT /api/menus/:id

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
nombre	string	Sí	Nuevo nombre del menú

Respuestas

Código	Descripción	Body
200	Actualización exitosa	{ "message": "Menú actualizado correctamente", "data": { "id": 1, "nombre": "..." } }
400	Falta el nombre	{ "error": "nombre es obligatorio" }
404	Menú no encontrado	{ "error": "Menú no encontrado" }

DELETE /api/menus/:id

Elimina el menú indicado. No requiere body.

Código	Descripción	Body
200	Eliminación exitosa	{ "message": "Menú eliminado correctamente" }
404	Menú no encontrado	{ "error": "Menú no encontrado" }

5. Reservaciones

Permite a usuarios autenticados crear y cancelar reservaciones de mesas. El sistema valida automáticamente que la mesa no esté ya reservada en el mismo horario antes de confirmar.

Resumen de endpoints

Método	Endpoint	Descripción	Rol requerido
POST	/api/reservations	Crear una reservación	Cualquiera
DELETE	/api/reservations/:id	Cancelar una reservación	Cualquiera

POST /api/reservations

Nota: El usuario se obtiene automáticamente del token JWT. No se envía el ID del usuario en el body.

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
id_restaurante	número	Sí	ID del restaurante donde se hace la reservación
id_mesa	número	Sí	ID de la mesa a reservar
fecha_hora	string	Sí	Fecha y hora en formato ISO 8601. Ej: "2027-06-15T19:00:00"
duracion	número	Sí	Duración de la reservación en minutos
cant_personas	número	Sí	Cantidad de personas (no debe exceder la capacidad de la mesa)

Respuestas

Código	Descripción	Body
201	Reservación exitosamente creada	{ "message": "Reserva creada correctamente", "id": 42 }
400	Faltan datos obligatorios	{ "error": "Faltan datos obligatorios" }
409	Mesa ya reservada en ese horario	{ "error": "La mesa ya está reservada en ese horario" }
401	No autenticado	{ "error": "No autorizado" }

DELETE /api/reservations/:id

Cancela la reservación indicada. No requiere body.

Código	Descripción	Body
200	Cancelación exitosa	{ "message": "Reserva cancelada correctamente" }
404	Reservación encontrada	{ "error": "Reserva no encontrada" }

6. Pedidos

Permite a usuarios autenticados realizar pedidos de platos. El sistema valida que el restaurante exista, que los platos pertenezcan a ese restaurante y que se haya enviado al menos un plato.

Resumen de endpoints

Método	Endpoint	Descripción	Rol requerido
POST	/api/orders	Crear un pedido	Cualquiera
GET	/api/orders/:id	Obtener detalle de pedido	Cualquiera

POST /api/orders

Nota: El usuario se obtiene automáticamente del token JWT. No se envía el ID del usuario en el body.

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
id_restaurante	número	Sí	ID del restaurante donde se hace el pedido
id_tipo_pedido	número	Sí	1 = para llevar, 2 = en mesa
platos	array	Sí	Lista de platos. Mínimo 1 elemento.
platos[].id_plato	número	Sí	ID del plato a pedir
platos[].cantidad	número	Sí	Cantidad del plato (mínimo 1)
descripcion	string	No	Instrucciones especiales o notas del pedido

Ejemplo de request

```
POST /api/orders
{
  "id_restaurante": 1,
  "id_tipo_pedido": 1,
  "platos": [
    { "id_plato": 3, "cantidad": 2 },
    { "id_plato": 7, "cantidad": 1 }
  ],
  "descripcion": "Sin picante por favor"
}
```

Respuestas

Código	Descripción	Body
201	Pedido creado exitosamente	{ "message": "Pedido creado correctamente", "id": 15 }
400	Datos incompletos o inválidos	{ "error": "Datos incompletos" }
400	Restaurante no existe	{ "error": "Restaurante 99 no existe" }
401	No autenticado	{ "error": "No autorizado" }

GET /api/orders/:id

Devuelve el detalle completo de un pedido, incluyendo todos los platos con sus precios y subtotales.

Respuesta 200

```
{ "id": 15, "id_usuario": 3, "id_restaurante": 1,
  "tipo_pedido": "para llevar", "descripcion": "Sin picante",
  "platos": [
    { "id_plato": 3, "nombre": "Pancakes", "cantidad": 2,
      "precio_unitario": 2500, "subtotal": 5000 },
    { "id_plato": 7, "nombre": "Sopa de Mariscos", "cantidad": 1,
      "precio_unitario": 4000, "subtotal": 4000 }
  ],
  "total": 9000 }
```

Respuestas de error

Código	Descripción	Body
401	No autenticado	{ "error": "No autorizado" }
404	Pedido no encontrado	{ "error": "Pedido no encontrado" }

7. Búsqueda

Endpoints del microservicio WS Search, accesibles via Nginx con el prefijo `/search/`. La búsqueda usa Elasticsearch con queries multi_match ponderadas: coincidencias en el nombre tienen mayor peso que en la categoría, y estas mayor peso que en la descripción.

Resumen de endpoints

Método	Endpoint	Descripción	Auth
GET	/search/products	Búsqueda textual de productos	No
GET	/search/products/category/:cat	Filtrar por categoría exacta	No
POST	/search/reindex	Reindexar catálogo completo	No

GET /search/products?q=texto

Parámetros de query

Parámetro	Tipo	Obligatorio	Descripción
q	string	Sí	Texto a buscar en nombre, categoría y descripción

Respuesta 200 — array de resultados ordenados por relevancia

```
[ { "id": 1, "nombre": "Casado", "categoria": "Típico",
  "descripcion": "Plato típico costarricense",
  "precio": 3500, "id_restaurante": 1, "score": 1.87 } ]
```

Si no hay resultados, devuelve un array vacío [].

Campos de respuesta

Campo	Tipo	Descripción
id	número	ID interno del plato
nombre	string	Nombre del plato
categoria	string	Categoría del plato
descripcion	string	Descripción. Si el plato no tiene, devuelve "Producto sin descripción"
precio	número	Precio en colones
id_restaurante	número	ID del restaurante al que pertenece
score	número	Relevancia del resultado. Mayor valor = más relevante

GET /search/products/category/:categoria

Filtra productos por categoría exacta. El parámetro va en la URL, no en la query string.

```
GET /search/products/category/Típico
GET /search/products/category/Pizza
```

Devuelve el mismo formato que la búsqueda textual. Array vacío si no hay productos en esa categoría.

POST /search/reindex

Reindexa todos los productos del catálogo en Elasticsearch leyendo desde la base de datos activa. Se llama también automáticamente al final del pipeline de Airflow. No requiere body.

Código	Descripción	Body
200	Reindexación exitosa	{ "message": "Reindexación completada", "total": 47 }
500	Error durante la reindexación	{ "error": "descripción del error" }

8. Grafos (Neo4J)

Endpoints del microservicio rutas-service para análisis del grafo Neo4J. Accesibles via Nginx con prefijo /grafos/ o directamente en <http://localhost:5050/grafos/>.

Resumen de endpoints

Método	Endpoint	Descripción	Auth
GET	/grafos/co-compras	5 pares de platos más comprados juntos	No
GET	/grafos/usuarios-influyentes	Usuarios por cantidad de recomendaciones	No
GET	/grafos/camino-minimo	Camino mínimo entre dos ubicaciones	No

GET /grafos/co-compras

Analiza el grafo para encontrar los cinco pares de platos que más frecuentemente se piden en el mismo pedido. Útil para implementar un sistema de recomendaciones tipo 'los que pidieron X también pidieron Y'.

Respuesta 200

```
[ { "producto1": "Pizza Pepperoni", "producto2": "Limonada Natural",
  "vecesJuntos": 42 }, ... ]
```

Campos de respuesta

Campo	Tipo	Descripción
producto1	string	Nombre del primer plato del par
producto2	string	Nombre del segundo plato del par
vecesJuntos	número	Cantidad de veces que ambos platos aparecen en el mismo pedido

GET /grafos/usuarios-influyentes

Devuelve los usuarios del sistema ordenados por la cantidad de otros usuarios que recomiendan, de mayor a menor. Las relaciones RECOMIENDA son simuladas durante la migración.

Respuesta 200

```
[ { "usuario": "María Pérez", "totalRecomendados": 3,
  "recomiendaA": ["Juan Mora", "Laura Jiménez", "Diego Rojas"] }, ... ]
```

GET /grafos/camino-minimo

Calcula el camino más corto entre dos zonas de Cartago usando el algoritmo Dijkstra del plugin Graph Data Science de Neo4J. Se puede optimizar por distancia o por tiempo.

Parámetros de query

Parámetro	Tipo	Obligatorio	Descripción
origen	string	Sí	ID de la ubicación de origen
destino	string	Sí	ID de la ubicación de destino

Parámetro	Tipo	Obligatorio	Descripción
criterio	string	No	distancia_km (default) o tiempo_min

Ubicaciones disponibles

ID	Nombre	Latitud	Longitud
cartago-centro	Cartago Centro	9.8647	-83.9192
tres-rios	Tres Ríos	9.9003	-83.9917
la-union	La Unión	9.9086	-83.9614
san-diego	San Diego	9.8891	-83.9456
paraíso	Paraíso	9.8358	-83.8650
el-tejar	El Tejar	9.8514	-83.9758
san-rafael	San Rafael	9.9200	-83.9300
agua-caliente	Agua Caliente	9.8750	-83.9050

Ejemplo de request

```
GET /grafos/camino-minimo?origen=cartago-centro&destino=tres-rios&criterio=distancia_km
```

Respuesta 200

```
{ "paradas": ["cartago-centro", "san-diego", "la-union", "tres-rios"],
  "costoTotal": 11.6 }
```

Respuestas de error

Código	Descripción	Body
400	Faltan parámetros origen o destino	{ "error": "Los parámetros origen y destino son obligatorios" }
404	No existe camino entre esas ubicaciones	{ "error": "No se encontró un camino entre esas ubicaciones" }

9. Rutas de entrega

Endpoint para asignar pedidos pendientes a repartidores y calcular el orden óptimo de visita. Accesible via Nginx con prefijo /rutas/ o directamente en <http://localhost:5050/rutas/>.

Resumen de endpoints

Método	Endpoint	Descripción	Auth
POST	/rutas/asignar	Asignar pedidos a repartidores con ruta optimizada	No
GET	/health	Health check del microservicio	No

POST /rutas/asignar

Lee los pedidos pendientes de la base de datos activa, les asigna una ubicación simulada de las 8 zonas de Cartago, los distribuye entre los repartidores con round-robin, y calcula el orden óptimo de visita para cada uno usando el algoritmo de vecino más cercano. Las distancias se consultan en Neo4J: primero conexión directa CONECTA, y si no existe se usa Dijkstra.

Request body (application/json)

Campo	Tipo	Obligatorio	Descripción
repartidores	array	Sí	Lista de repartidores disponibles. Mínimo 1.
repartidores[].id	string	Sí	Identificador único del repartidor
repartidores[].nombre	string	Sí	Nombre del repartidor
repartidores[].ubicacion_base	string	Sí	ID de la zona de partida del repartidor

Ejemplo de request

```
POST /rutas/asignar
{
  "repartidores": [
    { "id": "rep-001", "nombre": "Carlos Solano",
      "ubicacion_base": "cartago-centro" },
    { "id": "rep-002", "nombre": "Marcela Brenes",
      "ubicacion_base": "tres-rios" }
  ]
}
```

Respuesta 200

```
{
  "asignaciones": [
    {
      "repartidor": { "id": "rep-001", "nombre": "Carlos Solano",
        "ubicacion_base": "cartago-centro" },
      "ruta": [
        {
          "id_pedido": "P-1",
          "cliente": "María Pérez",
```

```

      "ubicacion_destino": "san-diego",
      "ruta_detallada": {
        "desde": "cartago-centro",
        "hasta": "san-diego",
        "paradas_intermedias": ["cartago-centro", "san-diego"],
        "distancia_km": 3.2,
        "conexion_directa": true
      }
    },
    ],
    "distanciaTotal": 14.5
  }
]
}

```

Campos de la respuesta

Campo	Tipo	Descripción
asignaciones	array	Una entrada por cada repartidor
asignaciones[].repartidor	object	Datos del repartidor
asignaciones[].ruta	array	Pedidos en orden óptimo de visita
asignaciones[].ruta[].id_pedido	string	ID del pedido
asignaciones[].ruta[].cliente	string	Nombre del cliente
asignaciones[].ruta[].ubicacion_destino	string	Zona de entrega
asignaciones[].ruta[].ruta_detallada.distancia_km	número	Km desde la parada anterior
asignaciones[].ruta[].ruta_detallada.conexion_directa	boolean	True si hay conexión directa en Neo4J
asignaciones[].distanciaTotal	número	Total de km que recorre el repartidor

Respuestas de error

Código	Descripción	Body
400	No se envió el array de repartidores o está vacío	{ "error": "Se requiere un array de repartidores con al menos un elemento" }
400	Un repartidor tiene campos faltantes	{ "error": "Cada repartidor debe tener id, nombre y ubicacion_base" }
200	No hay pedidos pendientes	{ "mensaje": "No hay pedidos pendientes para asignar", "asignaciones": [] }

10. Códigos de error comunes

Todos los microservicios siguen la misma convención de códigos HTTP. La Tabla siguiente resume los códigos que puede devolver cualquier endpoint del sistema.

Código	Significado	Cuándo ocurre
200	OK	La operación se completó exitosamente
201	Created	El recurso fue creado exitosamente (POST)
400	Bad Request	Faltan campos obligatorios, tipos incorrectos o datos inválidos
401	Unauthorized	No se envió token JWT, el token expiró o es inválido
403	Forbidden	El token es válido pero el rol del usuario no tiene permiso para esa acción
404	Not Found	El recurso solicitado no existe en el sistema
409	Conflict	Conflicto de datos — por ejemplo, mesa ya reservada en ese horario
500	Internal Server Error	Error inesperado en el servidor. Revisar logs del contenedor

11. Flujo típico de uso

La siguiente tabla muestra el orden recomendado de llamadas para un flujo de uso completo del sistema, desde el registro hasta la consulta de rutas de entrega.

Paso	Acción	Endpoint	Auth
1	Crear cuenta	POST /api/auth/register	No
2	Iniciar sesión y guardar access_token	POST /api/auth/login	No
3	Agregar Authorization: Bearer {token}	— (en todas las requests siguientes)	—
4	Ver restaurantes disponibles	GET /api/restaurants	Sí
5	Ver el menú de un restaurante	GET /api/menus/:id	Sí
6	Buscar platos por texto	GET /search/products?q=texto	No
7	Buscar platos por categoría	GET /search/products/category/:cat	No
8	Hacer una reservación	POST /api/reservations	Sí
9	Realizar un pedido	POST /api/orders	Sí
10	Ver el detalle de un pedido	GET /api/orders/:id	Sí
11	Cancelar una reservación	DELETE /api/reservations/:id	Sí
12	Ver recomendaciones de co-compra	GET /grafos/co-compras	No
13	Calcular camino entre zonas	GET /grafos/camino-minimo?origen=X&destino=Y	No
14	Asignar rutas a repartidores	POST /rutas/asignar	No